

The `constexpr` specifier for object definitions

Alex Gilding (Perforce UK)

Jens Gustedt (INRIA France)

2022-04-08

org:	ISO/IEC JCT1/SC22/WG14	document:	N2954
	... WG21 C and C++ liaison		P2576
target:	IS 9899:2023	version:	3
date:	2022-04-08	license:	CC BY (https://creativecommons.org/licenses/by/4.0/)

0.1. Abstract

C++ has supported translation-time definition of first-class named constants for over ten years, while C, for all types besides `int`, is still limited to using second-class language features, in particular macros, during translation. This puts C at a significant disadvantage in terms of being able to share the same features between runtime and translation, and in being able to assert truths about the program during translation rather than waiting to assert in a runtime debug build.

0.1. Summary of Changes

- N2954
 - Base on N2952
 - Restrict the feature to object definitions
 - Split the compound literal feature off to N2955
- N2917
 - recursion limits; no UB in C++; no new ODR; no call before definition; linkage; initializer order
 - wording for function definitions, avoid VLA side effects
 - wording for compound literals
 - split wording for different kinds of constant expression and propagate kind; add wording to null pointer constant
 - community comments, implementability
- N2851
 - original proposal

1. Introduction

C requires that objects with static storage duration are only initialized with constant expressions. The rules for which kinds of expression may appear as constant expressions are quite restrictive and mostly limit users to using macro names for abstraction of values or operations. Users are also limited to testing their assertions about value behaviour at runtime because `static_assert` is similarly limited in the kinds of expressions it can evaluate during translation. We propose to add a new (old) specifier to C, `constexpr`, as introduced to C++ in C++11. We propose to add this specifier to objects, and to intentionally keep the functionality minimal to avoid undue burden on lightweight implementations.

A previous revision also had this feature for functions, but WG14 was not in favor of this for inclusion to C23.

2. Rationale

Because C limits initialization of objects with static storage duration to constant expressions, it can be difficult to create clean abstractions for complicated value generation. Users are forced to use macros, which do not allow for the creation of temporary values and require a different coding style. Such macros - especially if they would use temporaries, but have to use repetition instead because of the constraints of constant expressions - may also be unsuitable for use at runtime because they cannot guarantee clear evaluation of side effects. Macros for use in initializers cannot have their address taken or be used by linkage and are truly second-class language citizens.

The same restriction applies to `static_assert`: a user cannot prove properties about any expression involving a function call at compile-time, instead having to defer to runtime assertions.

C does provide enumerations which are marginally more useful than macros for defining constant values, but their uses are limited and they do not abstract very much; in practice they are only superior in the sense that they have a concrete type and survive preprocessing. Enumerations are not really intended to be used in this way.

In C++, both objects and functions may be declared as `constexpr`, allowing them to be used in all constant-expression contexts. This makes function calls available for static initialization and for static assertion-based testing.

The subset of headers which are able to be common between C and C++ is also increased by adding this feature and strictly subsetting it from the C++ feature. Large objects can be initialized and their values and generators asserted against during translation by both languages rather than forcing a user to switch to C++ solely in order to get such assertions.

3. Proposal

We propose adding the new keyword `constexpr` to the language and making it available as a storage-class specifier for objects.

A scalar object declared with the `constexpr` storage-class specifier is a constant. It must be fully and explicitly initialized according to the static initialization rules. It still has linkage appropriate to its declaration and it exist at runtime to have its address taken; it simply cannot be modified at runtime in any way, i.e. the compiler can use its knowledge of the object's fixed value in any other constant expression.

3.1. Types

There are some restrictions on the type of an object that can be declared with `constexpr` storage duration. There is a limited number of constructs that are not allowed:

pointer types:

allowing these to use non-trivial addresses would delay the deduction of the concrete value from translation to link-time. For most of the use cases, such a feature can already be coded by using a `static` and `const` qualified pointer object, we don't need `constexpr` for that. Therefore we only allow pointer types if the initializer value is null.

variably modified types:

these can only occur if the declaration of an array size is not a constant expression. Since we want the feature to be completely determined at translation-time, `constexpr` VLA and derived types are non-sensical, here.

atomic types:

because objects that are declared with this may temporarily need access (or maybe even modify) an lvalue and impose sequentially consistent synchronization where only a translation-time value should be used and no lvalue should be accessed.

volatile:

It would not be clear what the semantics of a `volatile constexpr` object would be, for example if it could possibly change by means that are not under the control of the programmer.

restrict:

Similarly for `restrict`. The only pointer types that are allowed are null pointers and for them, `restrict` is useless.

Generally, it does not make sense to use any of the currently provided standard qualifiers on a `constexpr` object. For convenience we only allow `const` qualification, but which is redundant.

Other qualifiers may be introduced at a later time that might hold more meaning for these objects.

3.2. Aggregate or union types

In a previous version of this paper we also proposed relaxing the constant-expression rules to allow access to aggregate members when the object being accessed is declared as a `constexpr` object and (in the case of arrays) the element index is an integer constant expression. WG14 was not in favor of the proposed text.

3.2.1. Structure or union types

Nevertheless we observe that the member access operator `.` is not explicitly excluded from the admissible syntax of constant expressions (see 6.6 for a constraining list of exceptions), and that removing it from there might impact implementations that already allow structure or union types as an extension.

Thus we propose to maintain the status quo and to allow the `.` operator within constant expressions of all kinds. By the defaults that are already in place, a member of a `constexpr` structure or union inherits all properties from the structure or union. With the definitions that we propose the name of the member would still be an “identifier declared with `constexpr`” and thus be a named constant.

Union types here merit special consideration, because we don’t want to add new undefined behavior with this construct. A translator will always be able to deduce if the bit-pattern that is imposed for any union member by the initializer provides a valid value for the named constant.

Since this is a translation-time feature, the constraint in 6.6 p4

Each constant expression shall evaluate to a constant that is in the range of representable values for its type.

always kicks in, and forces a diagnostic if and when the implementation is not able to produce a consistent value for any member.

Note that allowing structure types agrees with C++’s policy, whereas also allowing it for union types is less constraining than for them. Here, C++ only allows to access the “active” member of a union in a constant expression. Since C does not have this concept of an active member of a union, and since type-punning through union is a distinguished feature in C, it is not easy to map this restriction to C.

3.2.2. Array types

The use of a `constexpr` array object in a context that requires a constant expression is not possible without special considerations, of which WG14 was not in favor for C23. Nevertheless we maintain the possibility to define such named constants because they still have other advantages over `const`-qualified arrays of static storage duration:

- The initializer must be composed of constant expressions. So even if the array elements are not constant expression by themselves, many optimizations will still be applicable to them under the as-if rule.
- The base type of the array is enforced to be `const` qualified and not `restrict`, `volatile` or `_Atomic` qualified.
- Each assignment expression in the initializer still has to provide a valid value for the type.
- A diagnostic can be expected if the initialization of an element at an excess position is attempted.
- The property for character arrays (even wide) being strings is easily maintained by the translator and diagnostics can be issued in circumstances that require strings, for example as arguments to formatted IO functions. More generally, diagnostics that are based on the content of such character arrays can be issued.

3.3. Linkage

We do not propose changing the meaning of the `const` keyword in any way (this differs between C and C++) - an object declared at file scope with `const` and without `static` continues to have external linkage; an object declared with static storage duration and `const` but not `constexpr` is not considered any kind of constant-expression, barring any implementations that are already taking advantage of the permission given in 6.6 paragraph 10 to add more kinds of supported constant expressions.

The difference between the behaviour of `const` in C and in C++ is unfortunate but is now cemented in existing practice and well-understood. Since changing the status of existing `const` qualified variables would implicitly change the status of derived array declarations, we would oppose changing that now.

The `constexpr` feature itself does not have this problem, because it can only be used through an explicit code change. Nevertheless, `constexpr` objects will typically be defined in header files, so we have to ensure that they don't create multiply-defined-symbol conflicts. Therefore, in accordance with C++, file-scope `constexpr` obtain internal linkage and block-scope no linkage at all.

3.4. Storage duration

For the storage duration of the created objects we go with C++ for compatibility, that is per default we have automatic in block scope and static in file scope. The default for block scope can be overwritten by `static` or refined by `register`. It would perhaps be more natural for named constants

- to be addressless (similar to a `register` declaration or an enumeration),
- to have static storage duration (imply `static` even in block scope), or
- to have no linkage (similar to `typedef` or block local `static`)

but we decided to go with C++'s choices for compatibility.

Also we don't allow thread local named constants

`thread_local`:

Because we only allow constant expressions as initializers for named constants, a split into one distinct object per thread does not make much sense.

4. Alternatives

C currently has only one class of in-language entity that can be defined with a value and then used in a constant context, which is an enumeration. This is limited to providing a C-level name for a single `int` value, but is extremely limited and is a second-class feature closer to macro constants than to C objects. These cannot be addressed and also cannot be used to help much in the composition of arbitrarily-typed constant expressions during translation.

5. Impact

As above, the existing incompatibility of `const` between C and C++ is preserved because the proposal does not intend to break or change any existing C code. Code that intends to express identical constant semantics for values in both C and C++ should start using `constexpr` objects instead.

This change improves C's header compatibility with C++ by allowing the same headers to make use of better compile-time initialization features. This increases the subset of C++ headers which can be used from C and does not impose any new runtime cost on any C program.

6. Implementation Experience

There is widespread implementation experience of `constexpr` as a C++ feature. Internally to the QAC team, we have experience fitting C++11 ruleset `constexpr` to the C constant evaluator. Our C frontend does not share this component with the C++ compiler, so we were able to compare and contrast which work was reasonable to import and which was not (i.e. we have implemented `constexpr` fully before). We felt that full C++20 ruleset `constexpr` was completely unreasonable (probably not controversial!), but that the C++11 rules, including `constexpr` functions, designed to buildup from a minimalist perspective, were not difficult for a single-person team to add to a C evaluator. Implementing just the `constexpr` object part (without functions) as proposed here in this paper even has an implementation complexity that is much lower.

7. Wording

The wording changes proposed here are based on N2952 that sets the basis for some of the syntactical specifications.

7.1. Keywords (6.4.1)

Add `constexpr` to the list of keywords in 6.4.1.

7.2. Declarations (6.7)

According to the outcome for N2953 use alternatives 1 or 3 from N2952 to make `constexpr` declarations underspecified.

Alternatives

A declaration such that the declaration specifiers contain no type specifier or that is declared with `constexpr` is said to be underspecified.

A declaration with `constexpr` is said to be underspecified.

7.3. Storage-class specifiers (6.7.1)

Add `constexpr` to the list of storage-class specifiers in 6.7.1 p1.

7.3.1. Constraints

Named constants might possibly have static or automatic storage duration, but no other restrictions to their storage duration should apply.

According to the outcome for N2953 change paragraph 2

2 At most, one storage-class specifier may be given in the declaration specifiers in a declaration, except that `thread_local` may appear with `static` or `extern` and `constexpr` may appear with `auto`, `register` or `static`.¹²⁷⁾

2 At most, one storage-class specifier may be given in the declaration specifiers in a declaration, except that `thread_local` may appear with `static` or `extern` and `constexpr` may appear with `auto` type, `auto`, `register` or `static`.¹²⁷⁾

As stated above the possible types for named constants should be constrained. Add a new paragraph and footnote to the end of the *Constraints* section.

An object declared with storage-class specifier `constexpr` or any of its members, even recursively, shall not have an atomic type or a type that is volatile or restrict qualified. The declaration shall be a definition, shall have an initializer and shall be such that all expressions^{FNT0)}, if any, are either constant expressions or string literals.^{FNT1)} If an object or subobject declared with storage-class specifier `constexpr` has pointer type, the implicit or explicit initializer value for it shall be a null pointer.^{FNT2)}

^{FNT0)} Such as an assignment expression in an initializer or in an array bound.

^{FNT1)} As a consequence a `constexpr` object does not have a variably modified type.

^{FNT2)} The named constant corresponding to an object declared with storage-class specifier `constexpr` and pointer type is a constant expression with value null, and thus a null pointer and an address constant. Even if it has type `void*` it is not a null pointer constant.

7.3.2. Semantics

Adapt the changed p6 as of N2952

6 Storage-class specifiers specify various properties of identifiers and declared features; storage duration (`static` in block scope, `thread_local`, `auto`, `register`), linkage (`extern`, `static` and `constexpr` in file scope, `typedef`), value (`constexpr`) and type (`typedef`). The meanings of the various linkages and storage durations were discussed in 6.2.2 and 6.2.4, `typedef` is discussed in 6.7.8.

Then add a new paragraph, a footnote and a note to the end of the *Semantics* section

An object declared with a storage-class specifier `constexpr` has its value permanently fixed at translation-time; if not yet present, a `const`-qualification is implicitly added to the object's type. The

declared identifier is considered a constant expression of the respective kind, see 6.6.

NOTE An object declared in block scope with a storage-class specifier `constexpr` and without `static` has automatic storage duration, the identifier has no linkage, and each instance of the object has a unique address obtainable with `&` (if there also is no `register` specifier) or no address at all (with `register`). Such an object in file scope has static storage duration, the corresponding identifier has internal linkage, and each translation unit that sees the same textual definition implements a separate object with an address of its own.

7.4. Constant Expressions (6.6)

To introduce terminology, we stipulate that being a constant expression is a property of the declared identifier, and not of the underlying object. Add a new paragraph 5' after paragraph 5

5' An identifier that is an enumeration constant, a predefined constant or that is declared with storage-class specifier `constexpr` and an object type is a named constant. For enumeration and predefined constants, their value and type are defined in the respective clauses; for `constexpr` objects, such a named constant is a constant expression with the type and value of the declared object.

These new kinds of constants then have to be added in the appropriate places. Change the following three paragraphs.

6 An integer constant expression¹²⁴⁾ shall have integer type and shall only have operands that are integer constants, ~~enumeration constants~~ named constants of integer type, character constants, ~~predefined constants~~, `sizeof` expressions whose results are integer constants, `alignof` expressions, and floating constants or named constants that are the immediate operands of casts. Cast operators in an integer constant expression shall only convert arithmetic types to integer types, except as part of an operand to the `sizeof` or `alignof` operator.

8 An arithmetic constant expression shall have arithmetic type and shall only have operands that are integer constants, floating constants, ~~enumeration constants~~ named constants of arithmetic type, character constants, ~~predefined constants~~, `sizeof` expressions whose results are integer constants, and `alignof` expressions. Cast operators in an arithmetic constant expression shall only convert arithmetic types to arithmetic types, except as part of an operand to a `sizeof` or `alignof` operator.

9 An address constant is a null pointer,^{FNT3)} a pointer to an lvalue designating an object of static storage duration, or a pointer to a function designator; it shall be created explicitly using the unary `&` operator or an integer constant cast to pointer type, or implicitly by the use of an expression of array or function type. The array-subscript `[]` and member-access `.` and `->` operators, the address `&` and indirection `*` unary operators, integer constant expressions, and pointer casts may be used in the creation of an address constant, but the value of an object shall otherwise not be accessed by use of these operators.^{FNT4)}

FNT3) A named constant of integer type and value zero is a null pointer constant. A named constant with pointer type and value null is a null pointer.

FNT4) Named constants with arithmetic type, including names of `constexpr` objects, are valid in offset computations such as array-subscripts or in pointer casts, as long as the expressions in which they occur form integer constant expressions. In contrast to that, names of other objects, even if `const`-qualified and with static storage duration, are not valid.

7.5. Linkage (6.2.2)

Named constants (constexpr objects) will typically be defined in header files, so we have to ensure that they don't create multiply-defined-symbol conflicts. Change the following paragraph

*3 If the declaration of a file scope identifier for an object contains any of the storage-class specifiers `static` or `constexpr` or for a function contains the storage-class specifier `static`, the identifier has internal linkage.*³¹⁾

8. Acknowledgements

Additional thanks to Martin Uecker and Joseph Myers for detailed wording improvement suggestions for previous revisions of this paper.